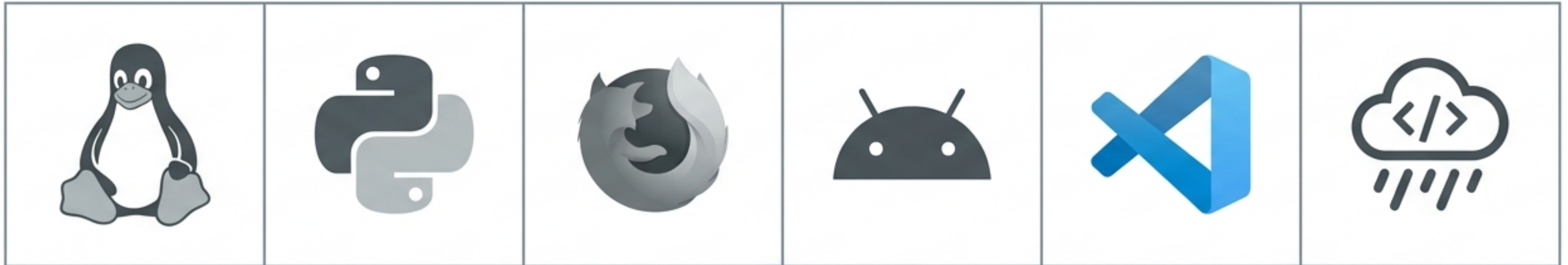


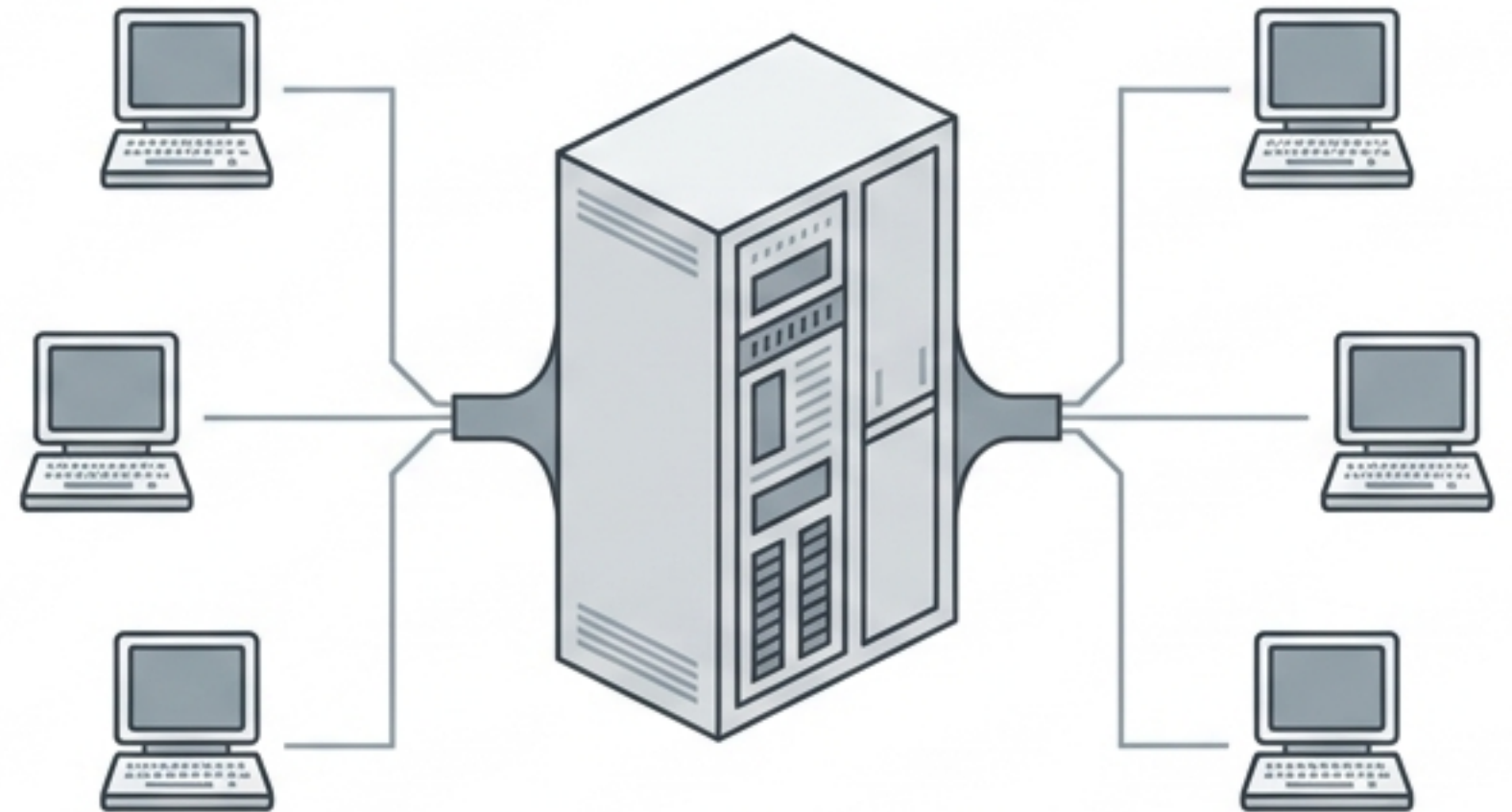
Colaborar en proyectos de programación

- El software casi nunca lo hace una sola persona. Muchos proyectos de programación los construye mucha gente a la vez, todos editando el mismo código.
- Ejemplos: Linux, Python, Firefox, Android, VS Code, WRF.



El mundo de los 70s

- El código vivía en un máquina central. Nadie tenía una copia del código.
- Cuando dos personas editaban el mismo archivo sin ninguna herramienta, los cambios se podían perder. Se necesitaba coordinación humana.
- Se hacían copias con fecha: `archivoX_fecha1.c` y `archivoX_fecha2.c`



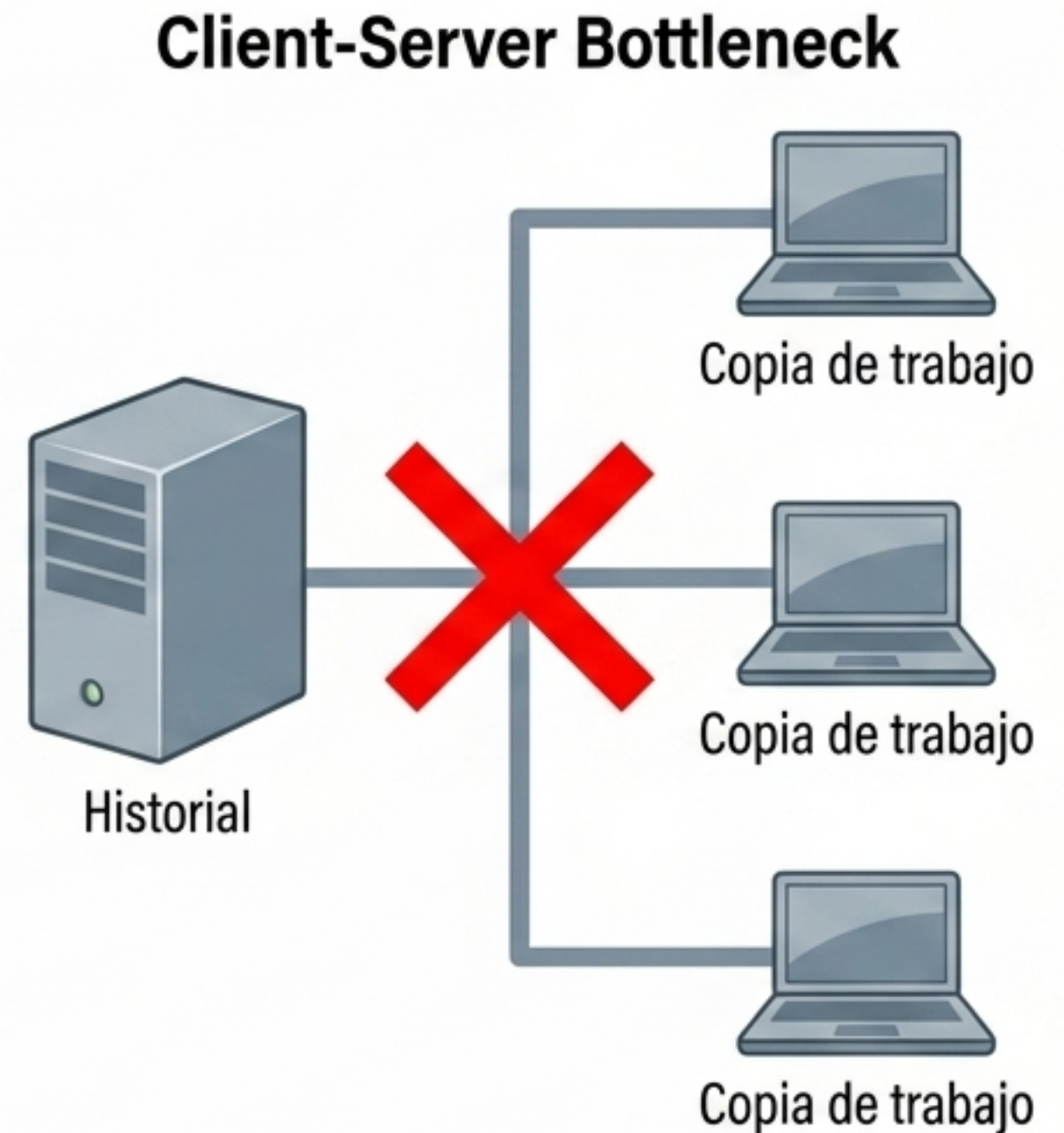
La primera solución: SCCS (1972)

- Creada por Marc Rochkind, en los Bell Labs, en 1972.
- Fue el primer sistema de control de versiones de la historia.
- **¿Cómo funcionaba?** Era local en el sentido de que no había red para intercambiar cambios, todos entraban en la misma máquina compartida donde vivían los archivos.
- **Revolucionario:** guardaba una versión más los cambios para reconstruir cualquier versión anterior. Cada versión tenía un identificador propio. Se podía volver atrás en el tiempo.
- Todos compartían los mismos archivos en la misma máquina, la solución para que no se pisaran fue impedir por diseño que dos editaran a la vez, mediante candados.



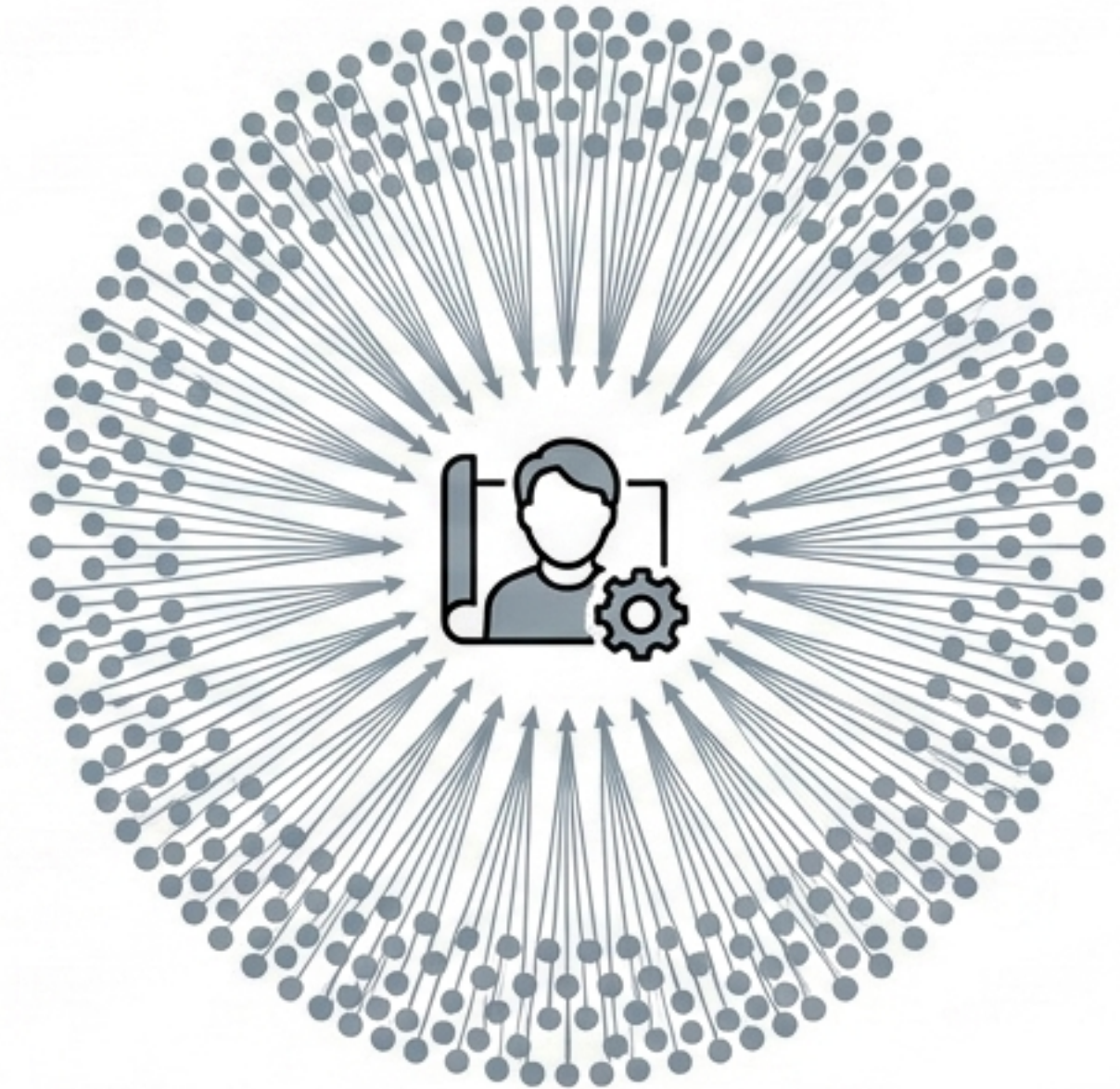
La evolución centralizada: SVN / Subversion (2000)

- Sucesor directo de CVS que arregló varios de sus peores defectos conservando el modelo centralizado.
- **¿Cómo funciona?** El historial completo del proyecto vive solo en el servidor central.
- **Copia de trabajo:** El programador tiene en su máquina local los archivos en su versión actual para editarlos, pero esta copia no contiene el historial ni las versiones viejas.
- **Dependencia de red:** Para ver el historial, comparar versiones o confirmar cambios (commit), es obligatorio estar conectado al servidor. Si el servidor cae, el equipo queda bloqueado.



Linus Torvalds y el kernel de Linux

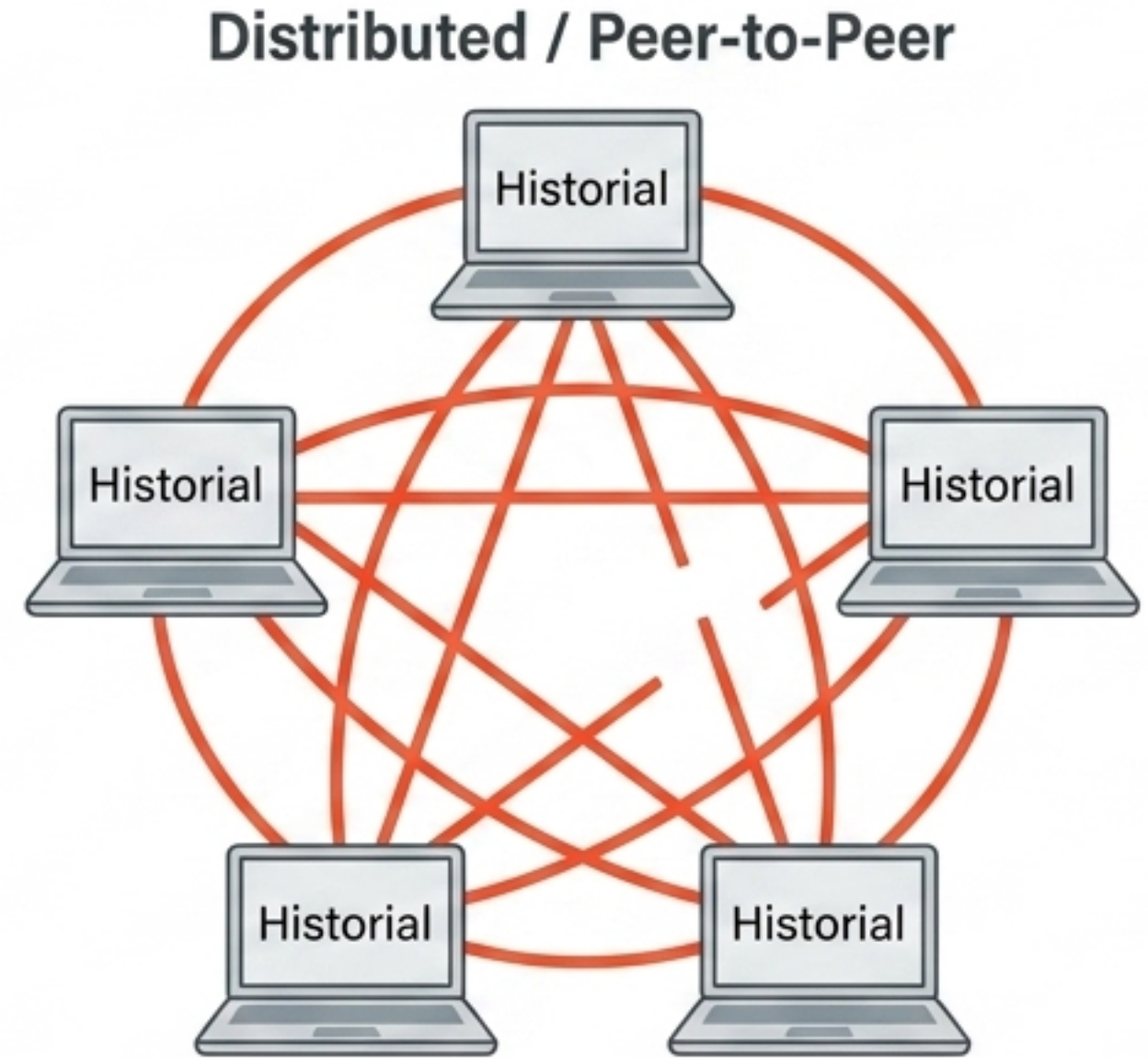
- Nació en Helsinki, Finlandia. Estudió ciencias de la computación.
- Es el creador de Linux. Lo publicó gratis y abierto, y con los años miles de personas empezaron a contribuir
- En algún momento, había miles de personas contribuyendo en el kernel del Linux a la vez. Torvalds necesitaba una herramienta a la altura.
- En 2002 el kernel adoptó **BitKeeper**, una herramienta distribuida. BitKeeper era de paga pero ofrecían una versión gratuita para proyectos de código abierto, bajo una condición: no podías analizar por dentro cómo funcionaba para hacer una copia o competencia.
- En 2005 se retiró la versión gratuita.



2002 - 2005

El nacimiento de Git (2005)

- Torvalds decidió no depender de otras herramientas y creó git.
- **Distribuido**: cada persona tiene una copia completa del proyecto. Puedes trabajar sin conexión, hacer **commits**, ver versiones anteriores y crear ramas sin depender de ningún servidor. Si el servidor central se cae, cualquier **copia local** tiene todo lo necesario para reconstruir el proyecto.
- **Velocidad**: como casi todo ocurre en tu propia máquina y no viajando a un servidor, las operaciones son casi instantáneas.



2005

Los pilares de Git: Ramas e Integridad

- Ramas baratas:

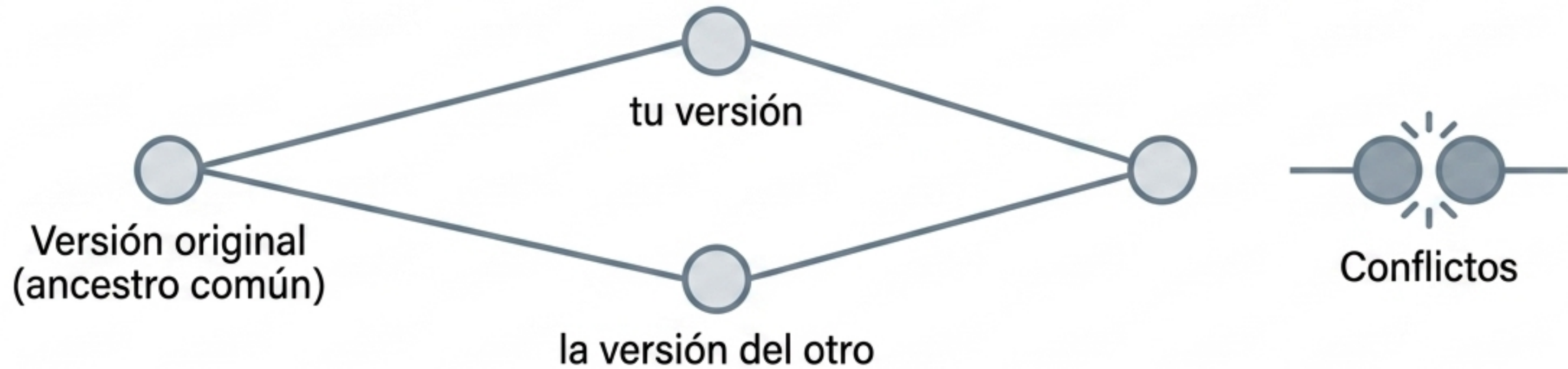
crear ramas para experimentar es tan rápido y ligero que que se volvió una práctica cotidiana. Permite probar ideas y fusionar solo lo que funciona.



- **Integridad:** cada cambio se identifica con una huella digital única (hash), por lo que cualquier corrupción es detectada inmediatamente por Git.



¿Cómo combina cambios Git? (Fusión de tres vías)



- **Fusión de tres vías (3-way merge):** Git mira tres puntos: la versión original (ancestro común de la que ambos partieron), tu versión y la versión del otro desarrollador para combinar cambios solos de forma automática si ocurren en partes diferentes del archivo.
- **Conflictos:** Ocurren únicamente cuando dos personas modifican la misma línea de forma distinta. Git detiene la fusión, marca las diferencias y te pide elegir a mano la versión final.

El nacimiento de GitHub (2008)

- Git era sumamente poderoso pero árido (se usaba solo desde terminal de comandos).
- En 2008 nace GitHub para ponerle a Git una capa amigable, visual y social.
- El gran aporte: El "Pull Request", una forma sencilla de proponer cambios y revisarlos antes de integrarlos (una función de la plataforma, no de Git nativo).
- Git es el motor, GitHub es el lugar de encuentro en la nube.



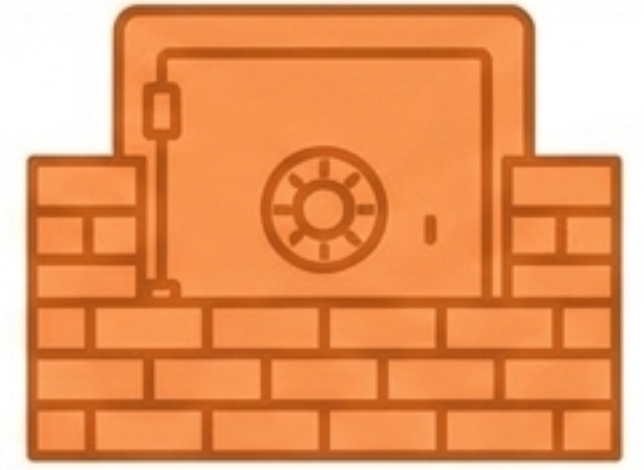
2008

GitLab (2011) y la Filosofía del Autohospedaje

- Creado en 2011 por Dmitriy Zaporozhets en Ucrania para colaborar con su equipo de forma abierta y accesible.
- Diferencia filosófica con GitHub: GitLab nació abierto y autohospedable; GitHub nació cerrado y primero-en-la-nube.
- GitLab permite correr de forma gratuita una plataforma de colaboración completa tipo GitHub dentro de tus propios servidores e infraestructura. Muy elegido por sectores con requisitos estrictos de seguridad de código (banca, gobierno, NASA).



en la nube



autohospedable



2011

Resumen: Git vs. Plataformas

Git: la herramienta local en tu computadora.	Compartir sin plataforma: requiere parches por correo o repositorios en servidores planos.	Plataformas en nube ajena: GitHub, GitLab.com.	Plataformas en servidor propio (autohospedado): GitLab auto-hospedado o GitHub Enterprise.

- Todas usan Git por debajo; solo cambia dónde vive el código y la interfaz de usuario.